Raritone Backend Architecture, Flow Diagram & Scaling Research

Nandini Bhimineni

Overview

The Raritone backend is designed as a scalable, realtime-ready, and AI-ready architecture for supporting ecommerce workflows, virtual try-on systems, media processing, and future AI integrations.

The backend is built using:

- Node.js
- Express.js
- MongoDB
- Socket.IO
- Cloudinary CDN

The architecture supports:

- Realtime communication
- Scalable APIs
- Optimized media delivery
- Secure backend workflows
- Future AI service integration

Backend Flow Diagram

Mobile App / Frontend

↓

Backend API

↓

AI Services

↓

Cloudinary CDN

↓

MongoDB Database

↓

Response Returned To Frontend

Backend Flow Explanation

Step 1 — User Interaction

Users interact through:

- Mobile application
- Frontend web application

Users can:

- Login/Register
- Upload images
- Request virtual try-on
- Manage products and profiles

Step 2 — Backend API Processing

The Express backend handles:

- API routing
- Request validation
- Authentication
- Media handling
- Database communication

Main backend modules:

- Controllers
- Routes
- Middleware
- Validators
- Models

Step 3 — AI Services Workflow

Future AI services will support:

- Virtual try-on processing

- Avatar generation
- Recommendation systems
- Body compatibility analysis

Planned AI workflow:

User Image + Product Data

↓

Backend Validation

↓

AI Processing Service

↓

Generated Try-On Result

↓

Cloudinary Upload

↓

Response To Frontend

Step 4 — Cloudinary Media Workflow

Cloudinary handles:

- Image storage
- CDN optimization
- Image transformations
- Compressed delivery

Workflow:

Image Upload

↓

Sharp Compression

↓

Cloudinary Upload

↓

Optimized CDN URL

↓

Frontend Delivery

Benefits:

- Faster image loading
- Lower bandwidth usage
- Scalable media delivery

Step 5 — MongoDB Workflow

MongoDB stores:

- Users
- Products
- Try-on history
- Orders
- Wishlist
- Avatar data
- Measurements

Benefits:

- Flexible schema structure
- Scalable document storage
- Easy AI data integration

Socket.IO Realtime Flow

Socket.IO provides:

- Realtime communication
- Live status updates
- Instant backend notifications

Example workflow:

Frontend Sends Try-On Request

↓

Backend Receives Event

↓

AI Processing Starts

↓

Realtime Status Updates Sent

↓

Frontend Receives Final Result

Benefits:

- Realtime user experience
- Reduced API polling
- Faster updates

Architecture Components

Frontend Layer

Handles:

- User interface
- API communication
- Image uploads
- Realtime socket events

Backend API Layer

Handles:

- Authentication
- Request processing
- Business logic
- Validation
- Security

AI Service Layer

Planned for:

- Virtual try-on generation

- Avatar creation
- Recommendation engines
- AI compatibility analysis

Database Layer

MongoDB stores:

- Structured user data
- Product data
- AI workflow history
- Media references

Media Layer

Cloudinary CDN handles:

- Optimized media delivery
- Image transformation
- Scalable storage

Security Architecture

Implemented security includes:

- JWT authentication
- Joi validation
- Helmet security
- Rate limiting
- Error middleware

Benefits:

- Protected APIs
- Secure user data
- Reliable backend communication

Scaling Research

Redis Caching

Redis can improve:

- API response speed
- Session storage
- Realtime scalability

Possible usage:

- Frequently accessed product data
- Session management
- AI processing cache

Benefits:

- Reduced database load
- Improved backend performance

Load Balancing

Load balancing distributes traffic across multiple servers.

Benefits:

- Reduced server overload
- Better uptime
- Scalable infrastructure

Useful for:

- Large-scale realtime systems
- High API traffic

Microservices Architecture

Future architecture can separate:

- Authentication service
- AI processing service
- Media service
- Realtime service

Benefits:

- Independent scaling
- Easier maintenance

- Improved performance

Queue Systems

Queue systems support:

- AI image processing
- Background tasks
- Async workflows

Examples:

- BullMQ
- RabbitMQ

Benefits:

- Prevents backend blocking
- Improves AI processing scalability

CDN Optimization

Cloudinary CDN optimization includes:

- Automatic compression
- Automatic format conversion
- Optimized image delivery

Benefits:

- Faster frontend loading
- Reduced bandwidth usage
- Scalable media delivery

MongoDB Atlas Scaling

MongoDB Atlas supports:

- Cloud database scaling
- Distributed clusters
- Backups
- Monitoring

Scaling strategies:

- Indexing
- Sharding
- Replica sets

Benefits:

- High availability
- Scalable database infrastructure

Progress Report

Completed Features

Implemented:

- Authentication APIs
- Product APIs
- Try-on APIs
- Wishlist APIs
- Order APIs
- Realtime Socket.IO setup
- Cloudinary integration
- Sharp compression
- Joi validation
- Helmet security
- Rate limiting
- Centralized error handling

Realtime Features

Completed:

- Socket.IO integration
- Realtime communication setup
- Backend event handling

Media Optimization

Completed:

- Image compression
- CDN optimization
- Optimized image delivery
- Cloudinary workflows

AI Workflow Preparation

Prepared:

- AI-ready APIs
- Avatar architecture
- Try-on workflow structure
- ProductMapping schema
- Realtime workflow planning

Future Improvements

Possible future enhancements:

- Redis implementation
- Distributed Socket.IO scaling
- GPU AI servers
- Recommendation engine
- AI queue systems
- Microservices deployment
- Realtime AI processing

Conclusion

The Raritone backend architecture provides a scalable and AI-ready foundation for:

- Realtime virtual try-on systems
- Optimized ecommerce workflows
- Secure API development
- Future AI integrations

The modular backend structure ensures:

- Maintainability

- Scalability
- Realtime communication support
- Future expansion capability